Relatório Técnico: Implementação de Montador Simplificado RISC-V

Tadeu Miller, Arthur Felipe

[1]Departamento de Ciência da Computação – Universidade Federal de Viçosa (UFV)
Campus Florestal – MG – Brasil

tadeu.andrade@ufv.br; arthur.f.campos@ufv.br

Abstract. *This report describes the development of a simplified RISC-V assembler in Python. The software converts assembly instructions into 32-bit machine code, supporting R, I, S, and B instruction formats, pseudo-instructions, and multiple numeric bases.*

Resumo. *Este relatório descreve o desenvolvimento de um montador RISC-V simplificado em Python. O software converte instruções assembly em código de máquina de 32 bits, suportando os formatos de instrução R, I, S e B, pseudo-instruções e múltiplas bases numéricas.*

1. Introdução

O objetivo deste trabalho prático é a implementação de um montador capaz de traduzir um conjunto de instruções da arquitetura RISC-V. Compreender o processo de montagem, bem como o software interage com o hardware é fundamental para compreender os computadores atuais [Patterson and Hennessy 2017]. O programa desenvolvido lê arquivos de texto contendo código fonte em assembly (extensão `.asm`) e gera as representações binárias, garantindo a compatibilidade com simuladores.

2. Instalações e Requisitos

O programa foi desenvolvido em **Python**. A escolha da linguagem se deu pela facilidade na manipulação das strings e dos dicionários. Para rodar o projeto em um ambiente Linux, siga os passos:

1. Verifique se o interpretador Python 3 está instalado na máquina:

```
    sudo apt update
    sudo apt install python3
```

2. A execução não exige a instalação de outras bibliotecas externas via `pip`, pois o código utiliza apenas a biblioteca nativa `sys`.

3. Guia de Execução

O script identifica como o usuário deseja receber a saída com base nos argumentos passados via terminal, através da função `main`:

3.1. Saída no próprio Terminal

```
python3 montador.py entrada.asm
```

3.2. Escrita em Arquivo

Para salvar o resultado em um arquivo, utilizamos o parâmetro `-o`. O código valida a presença do argumento para evitar erros de execução:

```
python3 montador.py entrada.asm -o saida.txt
```

4. Detalhamento da Implementação

A implementação seguiu o manual de instruções da arquitetura RISC-V [Waterman and Asanović 2024] e foi dividida nos módulos descritos a seguir.

4.1. Tratamento de strings

Utilizamos a função `replace` para remover vírgulas e parênteses, permitindo que o `split()` isole tudo em uma lista. A função `registrador_para_binario(reg)` é responsável por converter os nomes dos registradores (Exemplo: x3) em endereços binários de 5 bits.

4.2. Imediatos e várias bases

O suporte a múltiplas bases (decimal, hex e binário) é feito via `int(valor, 0)` [pyt 2026]. Para imediatos negativos, aplicamos um `AND` bit a bit para gerar o complemento de 2 correto em 12 ou 13 bits.

```
def imediato_para_binario(imediato, bits=12):
    numero = int(imediato, 0)
    binario = format(numero & ((1 << bits)-1), f'0{bits}b')
    return binario
```

4.3. Tipo de instrução S

O formato S (instruções de store) divide o imediato de 12 bits em dois campos não contíguos na instrução final: os 7 bits superiores ocupam os bits 31–25, e os 5 bits inferiores ocupam os bits 11–7.

```
imm_1 = imm_bin[0:7]    # bits 11 a 5 -> posicao 31-25
imm_2 = imm_bin[7:12]   # bits 4 a 0  -> posicao 11-7

binario_final = f'{imm_1}{rs2}{rs1}{funct3}{imm_2}{opcode}'
```

4.4. Tipo de instrução B

O formato B (instruções de branch, também conhecidas como os if/else de linguagens de alto nível) exige um certo "embaralhamento"dos bits dentro da instrução final. Resolvemos isso fatiando a string gerada pela função de imediato, acessando os caracteres pelos índices e juntando-os na ordem certa.

```
# Exemplo de reorganizacao dos bits para o Tipo B
imm_1 = imm_bin[0]     # bit 12
imm_2 = imm_bin[2:8]   # bits 10 a 5
imm_3 = imm_bin[8:12]  # bits 4 a 1
```

```
imm_4 = imm_bin[1]     # bit 11

binario_final = f'{imm_1}{imm_2}{rs2}{rs1}{funct3}{imm_3}{imm_4
    }{opcode}'
```

4.5. Pseudo-instruções

O montador tem suporte a pseudo-instruções para simplificar a programação:

- `mv rd, rs` → `add rd, rs, x0`.
- `li rd, imm` → `ori rd, x0, imm`.
- `bnez rs, offset` → `bne rs, x0, offset`.

4.6. Fluxo de Controle e Tratamento de Erros

A função `main` coordena a leitura e escrita de arquivos. Caso uma instrução resulte num binário inválido ou não seja reconhecida, o programa utiliza `sys.exit(1)`. Isso evita que os binários finais gerados (ou o arquivo final gerado) contenha lixo. Assim garantimos que o montador gera código funcional para um simulador RISC-V.

5. Resultados e validação

A validação consistiu em comparar as saídas geradas pelo montador com os padrões dos binários definidos para a arquitetura RISC-V, garantindo que cada campo (opcode, funct, registradores) estivesse na posição certa.

5.1. Testes por Formato

As Figuras abaixo demonstram a correta alocação dos campos e o fatiamento de bits para os formatos R, I, S e B.

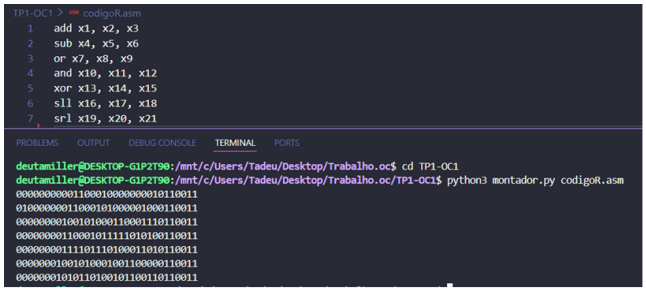

Figura 1. Saída binária no terminal para instruções do tipo R.

```
TP1-OC1 > codigoI.asm
addi x1, x2, 10
andi x7, x8, 25
lw x15, 8(x16)
lh x17, 12(x18)
lb x19, 4(x20)
ori x1, x2, 10

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc$ cd TP1-OC1
deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc/TP1-OC1$ python3 montador.py codigoI.asm
00000000101000010000000010010011
00000000110010100011100111100010011
00000000100010000010011110000011
00000000110010010001100010000011
00000000100101000001001100000011
00000000101000010110000010010011
```

Figura 2. Processamento de instruções do tipo I.

```
TP1-OC1 > codigoS.asm
sw x1, 8(x2)
sh x3, 12(x4)
sb x5, 4(x6)

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc$ cd TP1-OC1
deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc/TP1-OC1$ python3 montador.py codigoS.asm
00000000000100010010010000100011
00000000001100100010110001100011
00000000010110011000000100010011
```

Figura 3. Montagem de instruções Tipo S.

```
TP1-OC1 > codigoB.asm
beq x1, x2, 4
bne x3, x4, -4
beq x5, x6, 2046
bne x7, x8, -2048
beq x10, x11, 0
bne x12, x13, 1024

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc$ cd TP1-OC1
deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc/TP1-OC1$ python3 montador.py codigoB.asm
00000000001000001000001001100011
11111110010000011001111011100011
01111110011000101000111101100011
10000001000000111001000011100011
00000000101101010000000001100011
01000000110101100001000001100011
```

Figura 4. Montagem de instruções Tipo B.

5.2. Teste Geral

A Figura 5 demonstra a conversão de um arquivo com instruções mistas, validando a lógica do montador.

```
TP1-OC1 > teste.asm
lw x2, 0(x0)
sh x3, 12(x4)
sub x4, x2, x3
srl x19, x20, x21
xor x2, x2, x1
beq x1, x2, 4

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc$ cd TP1-OC1
deutamiller@DESKTOP-G1P2T90:/mnt/c/Users/Tadeu/Desktop/Trabalho.oc/TP1-OC1$ python3 montador.py teste.asm
00000000000000000010000100000011
00000000001100100001011000100011
01000000001100010000001000110011
00000001010110100101100110110011
00000000000100010100000100110011
00000000000100000100000001001100011
```

Figura 5. Teste simulando um código com instruções mistas.

6. Conclusão

A realização e construção do trabalho prático permitiu aprender de forma prática os conceitos vistos em sala de aula sobre as instruções da arquitetura RISC-V. Traduzir o assembly para binário deixou clara a importância de cada bit na formação do opcode, functs e registradores, além de mostrar como um montador funciona e conecta o software ao hardware.

O uso da linguagem Python se mostrou excelente, já que as funções nativas para lidar com strings ajudaram na etapa de limpeza das linhas e separação dos argumentos, enquanto a lógica de dicionários tornou o processo de busca das instruções muito mais fácil

do que uma estrutura cheia de ifs/elses. A principal complexidade durante o desenvolvimento foi a manipulação dos imediatos e o deslocamento de bits para o formato B, que exigiu um estudo de como a linguagem lida com representações binárias, complemento de dois em números negativos [pyt 2026].

Implementamos o reconhecimento de múltiplas bases numéricas (permitindo imediatos em binário ou hexadecimal, além do decimal) e suporte a pseudo instruções (como mv e li), conforme mencionado na seção de "Pontuação extra"na especificação do trabalho.

O link para o repositório do GitHub contendo o código do montador, casos de teste e README está disponível abaixo: https://github.com/arthurfcampos-png/oc-tp1

Referências

(2026). *Built-in Types: Bitwise Operations on Integer Types*. Python Software Foundation.

Patterson, D. A. and Hennessy, J. L. (2017). *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, Cambridge, MA, 1st edition.

Waterman, A. and Asanović, K. (2024). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V International. Version 20240411.